

Note that you're passing the `int()` function, not calling it, so you omit the parentheses after `int` in `collections.defaultdict(int)`. You can also pass `list` to use an empty list as the default value. Enter the following into the interactive shell:

```
>>> import collections
>>> booksReadBy = collections.defaultdict(list)
>>> booksReadBy['Al'].append('Oryx and Crake')
>>> booksReadBy['Al'].append('American Gods')
>>> len(booksReadBy['Al'])
2
>>> len(booksReadBy['Zophie']) # The default value is an empty list.
0
```

If you need a default value for every possible key, it's much easier to use `collections.defaultdict()` than use a regular dictionary and constantly call the `setdefault()` method.

Use Dictionaries Instead of a switch Statement

Languages such as Java have a `switch` statement, which is a kind of `if-elif-else` statement that runs code based on which one of many values a specific variable contains. Python doesn't have a `switch` statement, so Python programmers sometimes write code like the following example, which runs a different assignment statement based on which one of many values the `season` variable contains:

```
# All of the following if and elif conditions have "season ==":
if season == 'Winter':
    holiday = 'New Year\'s Day'
elif season == 'Spring':
    holiday = 'May Day'
elif season == 'Summer':
    holiday = 'Juneteenth'
elif season == 'Fall':
    holiday = 'Halloween'
else:
    holiday = 'Personal day off'
```

This code isn't necessarily unpythonic, but it's a bit verbose. By default, Java `switch` statements have “fall-through” that requires each block to end